

[1] AnalyticDB-V: A Hybrid Analytical Engine

Towards Query Fusion for Structured and Unstructured Data 총정리

저자: Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, Yuanzhe Cai (Alibaba Group) **학회/년도:** VLDB 2020 (Proceedings of the VLDB Endowment, Vol. 13, No. 12) **분량:** 약 14페이지 **역할군:** (A) VAQ 개념 원조; 본 논문 문제 정의의 출발점

요약

AnalyticDB-V는 Exqutor가 풀려는 문제, 즉 "벡터 검색과 SQL 분석 쿼리를 하나의 시스템 안에서 효율적으로 처리하는 것"을 최초로 산업 규모에서 구현한 시스템이다. 현대 데이터 파이프라인에서 구조화 데이터(테이블)와 비구조화 데이터(이미지, 텍스트 등)가 함께 존재하는데, 기존에는 이 두 종류를 별개의 시스템으로 처리해야 했다. AnalyticDB-V의 핵심 제안은 **벡터 유사도 검색을 SQL의 물리적 연산자로 구현하는 것**으로, 사용자가 하나의 SQL 문으로 벡터 검색과 관계형 분석을 동시에 표현할 수 있도록 한다.

시스템은 Lambda 아키텍처를 채택하여 스트리밍 레이어와 배치 레이어를 분리하고, VGPQ(Vector Graph Product Quantization)라는 새로운 ANN 검색 알고리즘을 제안한다. 가장 중요한 기여는 **정확도 인식 비용 기반 최적화(Accuracy-Aware Cost-Based Optimization)**로, ANN 검색의 본질적 근사성을 옵티마이저에 통합하여 정확도 요구사항과 비용 간의 트레이드오프를 관리한다. 다만 **카디널리티 추정** 문제를 깊이 다루지 않았으며, 이것이 Exqutor가 보완하는 핵심 빈틈이다.

본 논문과의 관계: VAQ(Vector-Augmented Analytical Query) 개념의 원형을 제시했으나, "벡터 검색 결과가 정확히 몇 건이나 나올지 추정하는 문제"는 미해결로 남겨두어, Exqutor의 근본적 동기를 제공한다.

상세분석

1.1 해결하고자 하는 문제

현대 데이터 파이프라인에서는 **구조화 데이터**(매출, 재고, 사용자 정보 등 숫자/텍스트 테이블)와 **비구조화 데이터**(이미지, 동영상, 오디오, 텍스트 문서 등)가 함께 생성된다.

기존에는 이 두 종류의 데이터를 별개의 시스템으로 처리했다: - 구조화 데이터 → 관계형 데이터베이스 (MySQL, PostgreSQL 등) - 비구조화 데이터 → 별도의 벡터 검색 엔진 (Faiss 등)

문제는 실제 비즈니스에서 두 데이터를 **함께** 분석해야 하는 경우가 매우 많다는 것이다. 예를 들어: - "이 상품 이미지와 비슷한 상품을 찾되, 가격이 100만원 이하이고 재고가 있는 것만 보여줘" - "이 얼굴 사진과 유사한 사용자를 찾아서, 그 사용자의 최근 구매 이력을 분석해줘"

이런 쿼리를 처리하려면 개발자가 두 시스템 사이에서 수동으로 데이터를 옮기고 결합해야 했다. 이 과정이 매우 복잡하고 느렸으며, 최적화 기회를 완전히 놓치게 된다.

1.2 핵심 아이디어: SQL로 하이브리드 쿼리 표현

AnalyticDB-V의 핵심 제안은 **벡터 유사도 검색을 SQL의 물리적 연산자(Physical Operator)로 구현**하는 것이다. 사용자는 하나의 SQL 문으로 벡터 검색과 관계형 분석을 동시에 표현할 수 있다.

```
SELECT product_name, price, similarity_score
FROM products
WHERE category = 'electronics'
      AND ANNS(image_embedding, query_vector, 10) -- 벡터 유사도 top-10
ORDER BY similarity_score DESC;
```

이렇게 하면 데이터베이스 옵티마이저가 벡터 검색과 필터링의 순서, 방식 등을 자동으로 결정해준다. 개발자는 "무엇을" 원하는지만 선언하면, 시스템이 "어떻게" 효율적으로 실행할지를 담당한다.

1.3 시스템 구조

AnalyticDB-V는 **Lambda 아키텍처**를 채택한다:

스트리밍 레이어(Streaming Layer): - 실시간으로 들어오는 벡터 데이터를 처리한다. - 새로 삽입된 벡터는 즉시 검색 가능하다. - 인메모리 임시 인덱스를 사용한다. - 예시; 새 상품 이미지가 업로드되는 순간, 즉시 유사 상품 검색에 포함된다.

배치 레이어(Batching Layer): - 주기적으로 새로 삽입된 벡터들을 압축하고, ANNS 인덱스를 재구축한다. - 대규모 데이터에 대해 최적화된 영구 인덱스를 관리한다. - 야간이나 유휴 시간대에 큰 배치를 처리하여 비용을 절약한다.

쿼리 처리 레이어: - 스트리밍 레이어와 배치 레이어의 결과를 병합하여 최종 결과를 반환한다. - 스트리밍 레이어에서 찾은 최근 벡터와 배치 레이어의 오래된 벡터를 함께 고려한다.

1.4 VGPQ (Vector Graph Product Quantization) 알고리즘

AnalyticDB-V의 핵심 기술적 기여 중 하나는 **VGPQ**라는 새로운 ANN 검색 알고리즘이다.

기존의 **IVF-PQ** (Inverted File with Product Quantization) 방식은: 1. 벡터 공간을 여러 클러스터(셀)로 나누고 2. 검색할 때 가까운 클러스터 몇 개만 방문하여 후보를 찾는다.

VGPQ는 여기에 **그래프 구조**를 추가한다: 1. 앵커(anchor) 중심점을 찾고 2. 그래프를 탐색하며 관련 서브셀(subcell)들을 효율적으로 선택하고 3. 후보 벡터들의 거리를 계산하여 결과를 반환한다.

이 방식은 대규모 벡터(수억~수십억 건)에서 기존 IVF-PQ보다 검색 정확도와 속도를 모두 향상시킨다. 특히 고차원 벡터(수백~수천 차원)에서 클러스터링 품질이 향상된다.

1.5 정확도 인식 비용 기반 최적화 (Accuracy-Aware Cost-Based Optimization)

가장 중요한 기여는 **ANNS 연산자를 데이터베이스 옵티마이저에 통합**한 것이다.

일반적인 데이터베이스 옵티마이저는 각 연산의 비용(I/O, CPU 등)을 추정하여 최적의 실행 계획을 선택한다. AnalyticDB-V는 여기에 **정확도(accuracy)**라는 새로운 차원을 추가한다.

ANN 검색은 본질적으로 근사적(approximate)이기 때문에, 파라미터 설정에 따라 정확도가 달라진다: - 파라미터를 높이면 → 정확도 높아지지만 비용(시간) 증가 - 예; 탐색할 클러스터 개수를 10개에서 100개로 늘리면, 정확도는 99%에서 99.9%로 개선되지만 시간은 2배 증가 - 파라미터를 낮추면 → 비용 줄지만 정확도 저하 - 예; 탐색할 클러스터 개수를 3개로 줄이면, 시간은 80% 단축되지만 정확도는 95%로 저하

옵티마이저는 사용자가 요구하는 정확도 수준을 만족하면서, 가장 비용이 적은 실행 계획을 선택한다. 만약 사용자가 "95% 정확도로 충분해"라고 명시하면, 시스템은 더 빠른 근사 알고리즘을 선택할 수 있다.

1.6 관계형 쿼리와 통합 메커니즘

ANNS 연산자를 관계형 대수에 통합하면서 중요한 문제들을 해결해야 한다:

카디널리티 추정의 어려움: 벡터 검색이 정확히 몇 건의 결과를 반환할 것인지를 미리 알기 어렵다. 예를 들어: - "가격 < 1000"인 상품을 찾으면 정확히 523개가 나온다 (카디널리티 = 523) - 하지만 "이 이미지와 유사한 상품"은 몇 개가 나올까? 이는 임계값 설정에 따라 100개~10,000개까지 가능하다.

최적화 기회의 발굴: 정확한 카디널리티가 있으면: - "벡터 검색을 먼저 할지, 가격 필터를 먼저 할지" 순서를 최적화할 수 있다. - "중간 결과가 작으면 Nested Loop Join, 크면 Hash Join"을 선택할 수 있다.

1.7 성능 평가

AnalyticDB-V는 아래와 같은 환경에서 평가되었다: - 벡터 데이터; 수천만 개의 상품 이미지 임베딩(512차원) - 구조화 데이터; 상품 메타데이터(가격, 카테고리, 재고 등) - 쿼리 유형; VAQ(벡터 + 필터 + 집계 혼합)

결과: - 순수 벡터 검색만으로는 최대 50배 성능 향상 - 벡터 + 관계형 쿼리 결합 시 최대 100배 향상 - 다양한 정확도 요구사항(90%~99.9%)에 대응 가능

1.8 본 논문과의 관계

AnalyticDB-V는 "벡터 검색을 SQL에 통합하자"는 아이디어를 제시했지만, **카디널리티 추정** 문제를 깊이 다루지 않았다. 즉, 벡터 검색 결과가 몇 건이나 나올지를 정확히 예측하는 문제는 남아있었다.

AnalyticDB-V에서의 가정: - 벡터 유사도 범위 검색(distance < D)의 결과는 **고정 비율**(예; 10%)로 추정 - 이 고정 가정은 실제 데이터 분포가 다를 때 매우 부정확함

Exqutor는 바로 이 빈틈을 메우는 연구이다: - AnalyticDB-V의 "고정 선택도" 가정을 버림 - **실제 인덱스를 탐색**하여 정확한 카디널리티를 얻음 - 이를 통해 옵티마이저가 더 나은 실행 계획을 세우도록 함

1.9 정확한 카디널리티의 가치: 구체적 예시

AnalyticDB-V의 "고정 10% 선택도" 가정이 얼마나 부정확할 수 있는지 보자:

```
쿼리: 상품 이미지 유사도 검색 + 가격 필터
SELECT product_id, price
FROM products
WHERE image_similarity(embedding, query_img) < 0.5
      AND price < 100000
```

AnalyticDB-V 옵티마이저의 추정:

- 총 상품: 1,000,000개
- 유사도 필터: 10% 선택도 가정 → 100,000개
- 가격 필터: 30% 선택도 (알려짐) → 300,000개
- 최종 결과 추정: $100,000 \times 0.3 = 30,000$ 개

하지만 실제 데이터:

- 유사도 필터 실제 결과: 0.5% (5,000개)
→ 고정 가정과 오류: 5배 차이!
- 최종 실제 결과: $5,000 \times 0.3 = 1,500$ 개
→ 추정 오류: 20배!

최적화 영향:

- 30,000건 기준으로 계획 선택 (해시 조인 선택)
- 1,500건이 나올 줄은 몰라서 비효율적 계획 선택
- 추가 비용: 리소스 낭비, 응답시간 증가

추가 제기 문제

1. **일반화의 어려움**: AnalyticDB-V는 Alibaba의 e-commerce 환경에 특화되어 설계되었다. 일반적인 OLAP 환경에서는:
2. 벡터 크기가 더 클 수 있고 (수천 차원)

3. 다양한 벡터 필드가 여러 개 존재하며
4. 복잡한 조인 구조가 필요할 수 있다

이러한 일반화된 환경에서 AnalyticDB-V의 비용 모델이 얼마나 잘 작동하는지는 명확하지 않다. 특히 **복잡한 다중 조인 쿼리**에서 벡터 연산의 위치를 어디에 배치할지 결정하는 문제는 더욱 복잡해진다.

1. **실행 계획의 적응성:** 고정된 정확도 임계값(예; 95%)으로 시스템을 설정하면, 쿼리가 달라졌을 때도 같은 임계값을 사용한다. 하지만 어떤 쿼리는 99% 정확도가 필요하고, 다른 쿼리는 90%로도 충분할 수 있다. 이런 동적 조정 메커니즘이 필요하지 않을까?
2. **벡터 인덱스 구축 비용:** VG PQ 알고리즘은 검색 성능은 개선하지만, 인덱스 구축 비용도 고려해야 한다. 데이터가 계속 추가되는 환경에서 배치 레이어의 인덱스 재구축이 너무 자주 발생하면 시스템 부하가 될 수 있다.